

AI Course

Capstone Project Final Report

For students (instructor review required)

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Bayan: On-Device Arabic Text Simplification for Readers with Dyslexia

تبسيط · Simplify

05/10/26

Cogni

Marwan Elamami
Abdulrahman Khengari
Ahmed Alaeb
Sanad Ali
Mohammed Thabet
Abdul Majid Mraied

Content

1. Introduction

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Members and Role Assignments
- 1.4. Schedule and Milestones

2. Project Execution

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Design

3. Results

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. User Interface
- 3.5. Testing and Improvements

4. Projected Impact

- 4.1. Accomplishments and Benefits
- 4.2. Future Improvements
- References

5. Team Member Review and Comment

6. Instructor Review and Comment

Abstract

Bayan is an Android app that rewrites a selection of Arabic text, from inside any app and entirely on the phone, to lower its *linguistic load*: shorter clauses and fewer hard words. Most of the work went into evaluation. On SAMER, the main human-written Arabic simplification corpus, copying the input scores 77.5 SARI, so we built **BayanBench** (1,981 items), which scores meaning with an open model that agrees with human raters (AUC 0.85) and simplification, restraint and correctness with code. Our best model, BayanSimplify-v0.3, decoded with four beams, outperforms every earlier model: it keeps the meaning in 82% of test outputs (earlier models at most 78.6%) and cuts 5.2 words from the longest clause, and raters found the rewrites it made easier 86% of the time. It is the app's large model, at 471 MB and about 1.2 s per sentence on a mid-range phone.

Bayan at a Glance

The problem. Long clauses, rare words and missing short vowels make Arabic text hard for readers with dyslexia, and the tools that simplify text send it to a server.

What we built. Bayan is an Android app. A reader selects Arabic text in any app and chooses تبسيط (*Simplify*); a simpler version appears over the page, written entirely on the phone and offline. Bayan can also add tashkeel (the short vowels) and read the text aloud.

The models. Released on Hugging Face (huggingface.co/Congi-libya) as **BayanSimplify**: BayanSimplify-v0.1 (AraT5v2 on SAMER), BayanSimplify-v0.2 and its compact BayanSimplify-v0.2-Fast (AraT5v2 and AraBART on a 22,733-pair mix), and BayanSimplify-v0.3 (AraT5v2 on our own corpus, the best). The app downloads BayanSimplify-v0.2-Fast by default and offers BayanSimplify-v0.3 as its large model.

What it achieves.

Question	Answer
Does the best model keep the meaning?	BayanSimplify-v0.3 with four beams, the app's large model, keeps it in 82% of test outputs; every earlier model, at most 78.6%
Is the text simpler?	Its longest clause is 5.2 words shorter; the app's model before it managed 1.9
Do people agree?	Team raters found BayanSimplify-v0.3 easier on 86% of the outputs it changed (BayanSimplify-v0.2: 60%) and chose it over BayanSimplify-v0.2 in 8 of 8 comparisons; the reader with dyslexia chose it in every comparison that included it
Is it safe?	Every number is kept and no negation is flipped; a sentence that would lose either is shown unchanged
Does it run on a phone?	The fast model shows the first word in 82–172 ms and takes 222 MB; the large model writes a sentence in about 1.2 s and takes 471 MB
Tashkeel?	Added without changing a letter: 6.91% of vowels wrong outside case endings, from a 4.8 MB model
Can the numbers be trusted?	The meaning scorer agrees with human raters (AUC 0.85), and every score comes from BayanBench, an open benchmark anyone on the team can re-run

Objectives. Lower linguistic load without losing meaning: partly met. Add tashkeel without changing letters: met. Run on a phone with a default within 250 MB: met. Evidence from readers: started, with one reader with dyslexia (subsection 4.2).

What comes next. A reading study with more readers with dyslexia, measuring reading time and comprehension, and faster beam decoding, so the best model's quality also arrives word by word.

1. Introduction

1.1. Background Information

Developmental dyslexia affects an estimated 11% of Arab primary-school children, pooled across 18 studies ($N = 30,243$) [1]. It impairs reading through phonological processing, and Modern Standard Arabic adds obstacles of its own. Short vowels are normally left unwritten, so one written form carries several readings: كتب is *kataba* (he wrote), *kutiba* (it was written) or *kutub* (books), and the reader must resolve it from context. Clitics attach directly to the stem, producing long words that resist chunking, and Arabic-speaking children with reading disabilities show early deficits in exactly this morphological layer [2]. Prose chains clauses with و and ف across whole paragraphs, so one sentence can exceed working memory.

The evidence on what helps is uneven, and we build on the parts that have been measured. More frequent and shorter words sped reading and improved comprehension for readers with dyslexia [3, 4]. Unpacking bound morphemes into separate words helped Arabic readers with reading disabilities and not their typical peers [5]. Texts simplified by splitting clauses into short sentences were read faster and understood better by beginning readers, most by the weakest of them [6]. Full diacritization is contested: it adds visual crowding [7], and its benefit for our readers has not been tested. We therefore treat *simplify first* as the core intervention and diacritization as an optional stage.

Related work. SAMER [10] is the main human-written Arabic simplification corpus: 15 novels rewritten at two easier levels, sentence by sentence. BAREC [11] labels 69,441 sentences with 19 reading levels but is not parallel; LLM-written [12] and human-verified [13] corpora add other domains. None was written for readers with dyslexia, and none teaches sentence splitting (subsection 3.2). Control tokens let one model rewrite for different audiences [15, 16, 17], and our strength tags follow that line. SARI [18], the field’s standard score, correlates only $r = 0.36$ with human judgements of simplicity [20] and misses inserted or substituted content [21]; reference-free reading-level estimates [22] and edit-level error taxonomies [23] track simplicity and meaning better. Our benchmark follows behavioural testing [24], construct-validity and reporting checklists [25, 26] and document-clustered intervals [27]. LLM judges favour text close to what they wrote [28], so we check ours against people, with agreement measured by Krippendorff’s α [29].

1.2. Motivation and Objective

Readers meet difficult Arabic inside other applications — a message, a news page, a PDF — not in a reading app. Bayan therefore appears where the text already is: selecting text on an Android phone offers تبسيط (*Simplify*) in the system selection menu, and the rewrite opens in a panel over the same app. Every model runs on the phone, so the reader’s text never leaves it and the tool works offline.

The data for this did not exist. SAMER is small, literary and teaches mostly word substitution; the other public sources are LLM-written or cover one domain. Dyslexia-targeted neural simplification exists for English [8], and the one Arabic dyslexia study with a language model used it as a tutor, not as a rewriter [9]. Our searches of 17 and 23 September found no system that combines Arabic, automatic rewriting to lower reading difficulty, and on-device inference. Three research questions organise this report:

RQ1 Does the standard reference-based evaluation measure what this task needs, and what

does a model learn from the main human-written corpus?

RQ2 Can a model lower the linguistic load of Arabic text (long clauses, hard words) without changing its meaning, and how much of that can we show with validated measures?

RQ3 Does it run on a mid-range phone at interactive latency, and what do quantisation and the deployed input format cost?

We set four objectives, assessed in subsection 4.1:

O1 A simplifier that beats copying the input on held-out sentences a human simplified, and that lowers linguistic load without losing meaning.

O2 Diacritics restored on the simplified text without altering its letters.

O3 The pipeline running on a mid-range Android phone at interactive speed, with a default simplifier within 250 MB.

O4 Evidence from readers with dyslexia, collected with informed consent.

Scope of the claim. Bayan addresses readability simplification and read-aloud, in any app. Dyslexia has other axes that we have not tested with readers: decoding (the app can add tashkeel, a setting off by default), visual layout (the app has typography settings), the auditory route, and fluency. No reader with dyslexia has used the app itself yet; one has rated its rewrites. We therefore claim a measured reduction of linguistic load and the means to measure it, not that Bayan helps readers with dyslexia.

Contributions. (1) An evaluation audit: copy-heavy test sets make SARI reward inaction, and 9.3% of a public Arabic simplification corpus overlaps the SAMER and BAREC test sets. (2) BayanBench, a benchmark of behaviours with an open meaning scorer checked against human raters. (3) A small meaning judge that catches planted errors at 30 times its teacher's speed. (4) BayanSimplify-v0.3, a model that keeps meaning more often than every earlier one while simplifying more, running on the phone inside an Android app.

1.3. Members and Role Assignments

Every member merged work into the team repository; the evidence column cites the pull requests and issues that hold it (#).

1.4. Schedule and Milestones

The work ran from 13 September to 5 October 2026, ahead of submission on 5 October and the pitch on 6 October, in three build-and-measure cycles. BayanSimplify-v0.1 was trained, scored on locked test sets and diagnosed. The v0.2 models were designed from that diagnosis, trained on two architectures, run on a phone and scored on a benchmark built for the task. The last week replaced the benchmark's closed meaning judge with an open scorer checked against human ratings, and BayanSimplify-v0.3, trained on a second corpus and decoded with beam search, became the app's large model on 5 October (Figure 1, Table 2).

Table 1. Members, roles, and the work each delivered.

Member	Role	Delivered (evidence)
Marwan Elamami	Team lead; integration; on-device model	Product-form decision (#9); evaluation protocol and fixes (#16, #18); BayanSimplify-v0.1 diagnosis; data acquisition and leakage filtering; v0.2 recipe, data and task specifications (#27–#34); on-device engine and app; phone test; beam search and tashkeel in the app (#62, #66, #67); BayanBench and its v2 package (#48); rating site and analysis; Action Plan (#25); this report
Abdulrahman Khen-gari	Data lead	Validators for generated pairs (#10, #11); CAMEL readability check (#17, #20); synthetic corpus v0 of 13,072 pairs, split and published (#8, #38, #39); corpus v1 with the Gemma generator and Qwen judge, and its calibration (#44, #45); the meaning judge (#47, #49); BayanSimplify-v0.3: training, preference tuning and the int8 bundle (#48, #53); BayanBench v2 design and the open meaning scorer runs (#48); the beam-search proposal (#61); the benchmark report of 35 systems (#57, #58); the pitch deck (#50)
Ahmed Alaeb	Model training	SAMER conversion; BayanSimplify-v0.1 trained and documented (#5, #22); BayanSimplify-v0.2 (AraT5v2) trained (#28, #40); human ratings (#48)
Sanad Ali	Diacritization; co-training; presenter	Four-system diacritization benchmark; Libtashkeel selected (#6, #23); BayanSimplify-v0.2-Fast (AraBART) and the speed benchmark (#29, #35); human ratings (#48); presents the pitch
Mohammed Thabet	Android application	PROCESS_TEXT plugin, result sheet and 41 tests (#14, #21); model download pipeline, sheet states and typography settings (#31, #37); human ratings (#48)
Abdul Majid Mraied	Evaluation	Scoring and leakage scripts (#7, #15); locked-set converter, copy baselines, generate-and-score path (#19, #24, #26)

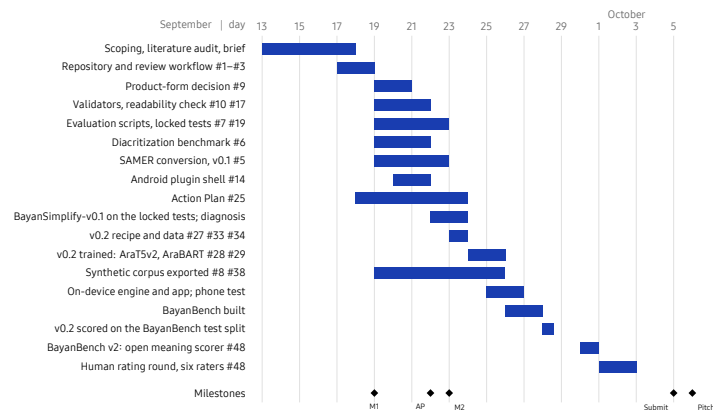


Figure 1. Timeline as executed, 13 September to 3 October. AP = Action Plan; M1, M2 = mentoring sessions. Numbers after a task are its GitHub issues.

Table 2. Milestones: target against actual.

Milestone	Target	Actual
Evaluation scripts ready (#7)	Mon 21 Sep	Mon 21 Sep
Diacritizer selected (#6)	Mon 21 Sep	Mon 21 Sep
Action Plan submitted	Tue 22 Sep	Tue 22 Sep
BayanSimplify-v0.1 scored on the locked test sets	Wed 23 Sep	Wed 23 Sep
v0.2 models trained, both architectures (#28, #29)	Fri 25 Sep	Fri 25 Sep
Model running on a phone, latency measured	Fri 2 Oct	Sat 26 Sep
BayanBench v2: open meaning scorer; all systems scored (#48)	—	Wed 30 Sep
Human rating round complete, six raters (#48)	Fri 2 Oct	Sat 3 Oct
BayanSimplify-v0.3 ship decision (subsection 3.3)	Sun 4 Oct	Mon 5 Oct: large model
Beam search and tashkeel in the app (#62, #66, #67)	—	Mon 5 Oct

2. Project Execution

2.1. Data Acquisition

We acquired five corpora and produced two more (Table 3). SAMER is used under written approval from CAMEL Lab to fine-tune and to publish the resulting weights non-commercially; the corpus itself is never redistributed, and no SAMER sentence is reproduced in this report. Baseet is used with its authors’ permission. Two test splits, SAMER test and BAREC test, were locked before any training.

Table 3. Corpora. Sizes are rows in the files we hold. Public means on the Hugging Face Hub, where the BayanSimplify models and their on-device bundles are public too (CC BY-NC 4.0).

Corpus	Size	Licence	Used for
SAMER [10]	14,343 / 2,983 / 3,277 pairs (train / dev / test)	research; written approval	BayanSimplify-v0.1 and the v0.2 models; locked test set
BAREC [11]	54,845 / 7,310 / 7,286 sentences	CC BY-SA 4.0	locked test set; BayanBench core items; generation sources
Baseet [12]	34,294 sources, three LLM-written levels each	with permission	the v0.2 models (6,888 meaning-filtered pairs)
DAASI [13]	1,509 pairs + 350 held out	CC0	the v0.2 models (1,502 pairs); out-of-domain test
Tashkeela [14]	743 test sentences	open	diacritization benchmark
Bayan synthetic v1	14,975 rows; Gemma 4 31B generator, Qwen judge	CC BY-SA 4.0, public on the Hub	current corpus; trains BayanSimplify-v0.3
Bayan synthetic v0	13,072 pairs; DeepSeek generator	CC BY-SA 4.0, public (Hub tag v0)	superseded; not used to train any model

The synthetic corpus, v0. Our first pipeline drew five DeepSeek V4.1 Flash candidates per BAREC train or dev source and accepted one only if an LLM equivalence validator and the CAMEL BAREC readability model both passed it. Its audit found tashkeel in 56.5% of rows, 63.4% of cut-off sources “completed”, and a validator that accepted 34% of planted added sentences, so we never trained on it and kept it online, tagged v0, with these defects stated.

The synthetic corpus, v1. Corpus v1 replaces it. Gemma 4 31B writes the targets from a stricter prompt, a calibrated Qwen judge checks meaning at a threshold of 0.75, and a target must lead its source by at least two readability levels. The 14,975 rows have no violation of the rule checks, and an audit of 1,039 pairs found 1.7% with a changed meaning. It trains BayanSimplify-v0.3.

2.2. Training Methodology

Task and base models. Given a source sentence x and a strength tag t , the model produces a rewrite $\hat{y} = f(t \oplus x)$ that keeps the meaning of x and is easier to read; training minimises token-level cross-entropy. We considered three pretrained encoder–decoders (Table 4): AraT5v2-base-1024 [32, 33, 35] (368M parameters, 46% of them in two untied embedding matrices), AraBART [34, 36] (139M) and an Arabic HPLT T5 [37, 38]. AraT5v2 came first because it beat AraBART at all three levels in the published Baseet baselines [12]; AraBART, 2.6 times smaller, is the on-device candidate. Neither vocabulary was built for diacritized text (1.6% of AraT5v2’s pieces carry a diacritic, and AraBART maps every mark to an unknown token), so every model receives its source with tashkeel and tatweel removed, in training and in the app.

Table 4. Candidate base models. “Tashkeel” is measured on 1,000 diacritized BAREC dev sentences: vocabulary entries containing a diacritic, and how many more tokens the same sentences cost with diacritics than without.

	AraT5v2-base-1024	AraBART	HPLT T5 base (Arabic)
Family	T5 [35]: span-corruption pretraining, relative position buckets	BART [36]: denoising pretraining, learned positions	T5 in the NorT5 variant [37]
Layers (enc/dec), width	12/12, $d=768$, 12 heads, gated-GELU FFN 2048	6/6, $d=768$, 12 heads, GELU FFN 3072	12/12, $d=768$, 12 heads, FFN 2048
Parameters	368M (46% in two untied $110k \times 768$ embeddings)	139M	$\approx 294M$
Vocabulary	110,208 SentencePiece	50,002 SentencePiece BPE	32,768
Context	1,024 tokens	1,024 positions	512 positions
Pretraining	Arabic MSA and dialects [32, 33]	large Arabic corpora [34]	HPLT v3 web crawl, Arabic subset [38]
Tashkeel	1.6% of pieces; $2.57 \times$ tokens	0 pieces; every mark becomes <unk>; $4.53 \times$ tokens	0 pieces; $1.81 \times$ tokens
Licence	none stated on the model card	Apache-2.0	Apache-2.0
Used in	BayanSimplify-v0.1 and the v0.2 models	BayanSimplify-v0.2-Fast	not trained

BayanSimplify-v0.1. AraT5v2 fine-tuned to rewrite SAMER’s level-5 sentences into their level-3 simplifications, with the prefix : بسط (*simplify*): AdamW at 10^{-4} , effective batch 16, 10 epochs, seed 42; unchanged pairs (42% of SAMER train) capped at 20%, and the checkpoint with the best SARI on a 500-row dev subset kept.

BayanSimplify-v0.2 and BayanSimplify-v0.2-Fast. The diagnosis of BayanSimplify-v0.1 (subsection 3.3) found three problems, and the v0.2 recipe changes one thing for each:

- Inaction was rewarded for the wrong reason.* All 6,033 unchanged SAMER pairs are kept, so the model learns when not to edit, and the checkpoint is selected on SARI over dev rows a human changed.
- SAMER is novels, and teaches word substitution.* DAASI’s everyday and administrative text and Baseet’s educational text are added. Baseet’s targets are LLM-written and often summarise, so a pair is kept only if it passes a *meaning filter* (LaBSE similarity ≥ 0.646 , at least 67% of the source’s length, every number kept), each threshold the 5th percentile of the same measure over DAASI’s human-verified rewrites.
- One strength does not fit every reader.* Every source carries a tag naming the style it was learned from (Table 5), so at run time one model serves light, medium and strong rewrites [15, 16].

Both architectures were trained with one shared script, so the architecture is the only difference: 10 epochs, batch 8 with 2 accumulation steps, fp16, seed 42, learning rate 10^{-4} (AraT5v2) or $5 \cdot 10^{-5}$ (AraBART, whose prefix : بسط drops the shadda its tokenizer cannot read). The script disables length-grouped batching, which in transformers 4.46 also shuffles the evaluation set, and stops if predictions and references fall out of order.

BayanSimplify-v0.3. The measurements of the v0.2 models named restraint, and protection of text that must not change, as the open problems. BayanSimplify-v0.3 retrains AraT5v2 on

Table 5. The v0.2 training set (22,733 pairs). No source overlaps any test set.

Tag	Taught by	Pairs	Meaning at run time
[S0]	SAMER L5→L3, all pairs (8,310 changed, 6,033 unchanged)	14,343	minimal: literary word swaps
[S1]	Baseet level 3, meaning-filtered	3,508	light
[S2]	Baseet level 2, meaning-filtered	1,954	medium (evaluated here)
[S3]	Baseet level 1, meaning-filtered	1,426	strong
[SA]	DAASI train	1,502	everyday and administrative style

corpus v1, runs behind the app’s code guards (subsection 2.4) and decodes with four beams (subsection 3.3). Figure 2 shows how each model’s measured problem shaped the next.

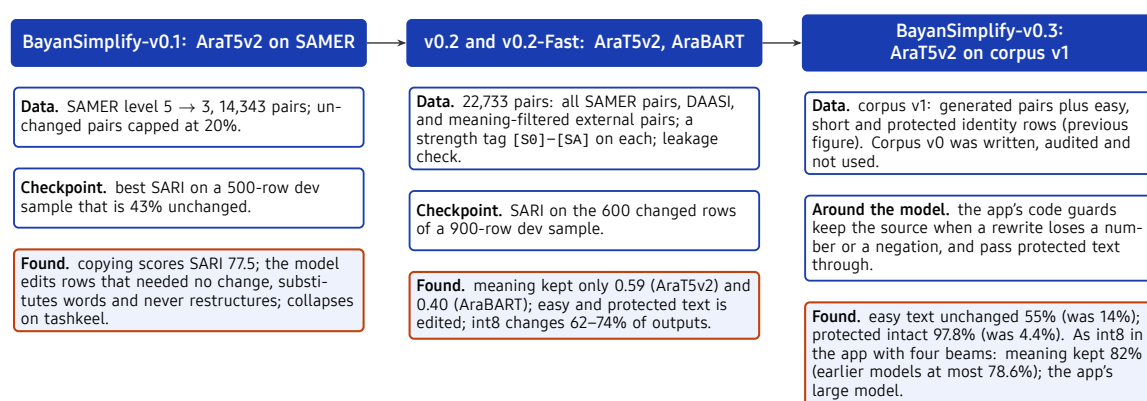


Figure 2. How the training pipeline evolved. Each model’s *Found* box is the measurement that produced the next model’s changes.

Evaluation: BayanBench. SARI measures agreement with one editor’s rewrite. A reader needs something else: a selection that comes back *easier* and *with the same meaning*, and text that should not change left alone. No public benchmark tests this for Arabic, so we built one against the checklists of [25, 26]. Every output is scored on separate measures, never combined into one number (Table 6): *meaning kept*, an output whose P(same meaning) from the scorer below is at least 0.5 and that keeps every number of its source; *simpler*, as words cut from the longest clause (sources with a clause over 15 words), reading levels lowered (CAMEL BAREC readability model) and hard words removed (SAMER lexicon levels 4–5); *behaviour*, as easy text left unchanged, protected text keeping its letters and a cut-off selection keeping its ending; and *flags* for a changed count of negations, limits or conditions. A clause is a span between punctuation marks, so the clause measure is always read next to meaning.

The 1,981 items form four sets reported side by side: *core* (BAREC test documents), *held-out* (news and legal text), *outside* (344 sentences from UN News, WHO, new Wikipedia articles, UN-HCR and Android help, published after every corpus we use) and *written* (50 medicine-leaflet sentences). Items are split by document (731 dev items from 129 documents, 1,250 test items from 197), and any BAREC document with a sentence in any training data was dropped whole. Meaning is scored by Gemma 4 31B [30] in vLLM [31], as a scorer rather than a writer: it sees the source and the output, never the system’s name, and the probability of yes to four questions (same meaning; a fact added; a fact dropped; a contradiction) is read from one forward pass. It replaces the closed Gemini 3.1 Pro judge of the first version, because anyone can re-run it and its graded score can be checked against people (about \$0.12 for 2,700 pairs); the 0.5 threshold was set before the human ratings and kept after them. Scores are shared data keyed by the (source, output) pair, an unscored output is reported as pending rather than guessed,

Table 6. BayanBench tracks, with items per track in the dev / test splits.

Track	Behaviour	Measured by	Items
A1–A3	meaning kept: general; hard spots (negation, numbers, conditions, cause, quantity, certainty); high stakes (legal, medical)	meaning scorer; numbers kept; flags	299 / 508
B	long clauses shortened	words cut from the longest clause; meaning scorer	104 / 187
B2, B3	fewer hard words; lower reading level	SAMER word levels; CAMEL BAREC readability model	across items
C	easy text left unharmed	unchanged; meaning scorer	129 / 202
D	correct Arabic	house punctuation rules; human ratings	across items
E1–E4	real selections: cut-offs, messy text, 40+ words, several sentences	end mark; clause length; meaning scorer	139 / 263
F	protected text (Qur’an, Bible, classical poetry)	letters unchanged (never sent to the scorer)	60 / 90
G	on the phone	int8 bundle against the same model in PyTorch; size, time	—
H	people	290 blind rating tasks, six raters (subsection 2.2)	—

and the v0.2 training data contains nothing a Gemma model wrote or approved, which limits preference leakage [28]. Every rate carries a 95% interval from a bootstrap over whole documents (2,000 resamples) [27], and a comparison counts only when the interval of the paired difference excludes zero. The benchmark ships as a Python package with its data and scores in a checksum-pinned Hugging Face dataset, so anyone on the team can rerun it.

Human rating protocol. To check the scorer, raters saw 245 single outputs and 45 comparisons from the six v0.2 systems on the dev split (64 documents), blind to the system. For each output they answered *meaning* (same, a minor change, a major change), *Arabic* (correct, a minor or a major error) and *ease* for a reader with dyslexia (easier, the same, harder); for a comparison, which version was better for such a reader. 210 outputs were drawn evenly per system and 35 where the scorer was least sure; 80 went to every rater, to measure agreement. Five team members rated on a web page built for the round. A sixth rater, a reader with dyslexia, gave informed consent and answered only the ease question, worded for them as هل كانت أسهل عليك؟ (was it easier for you?). An output’s meaning counts as kept when a strict majority of its raters said *same*.

The meaning judge. For screening data and checkpoints we distilled the 31B scorer into a four-head Gemma 4 E2B judge, released on the Hub in three variants (bf16, w8a8 and w4a16). At a 5% false-reject rate it flags 95.5% of planted errors and 86.5% of changes humans labelled as meaning changes, about 94% of the teacher’s score at about 30 times the speed. The 31B model remains the reference scorer in BayanBench.

2.3. Workflow

Each task is a GitHub issue with an owner, a reviewer and a definition of done; changes arrive as reviewed pull requests, with merged work from every member. SAMER text and model outputs on SAMER move only through private Kaggle datasets and a private Hugging Face organisation,

never through the repository. Figure 3 shows the data path and its three gates. The leakage check compares every training file with both locked test sets, by exact match and by character 5-gram Jaccard overlap of 0.8 or more; the meaning filter admits LLM-written pairs only if they keep the source’s meaning; generated pairs pass validators before release. The locked test sets are pinned by row count and SHA-256 and re-verified before every test run.

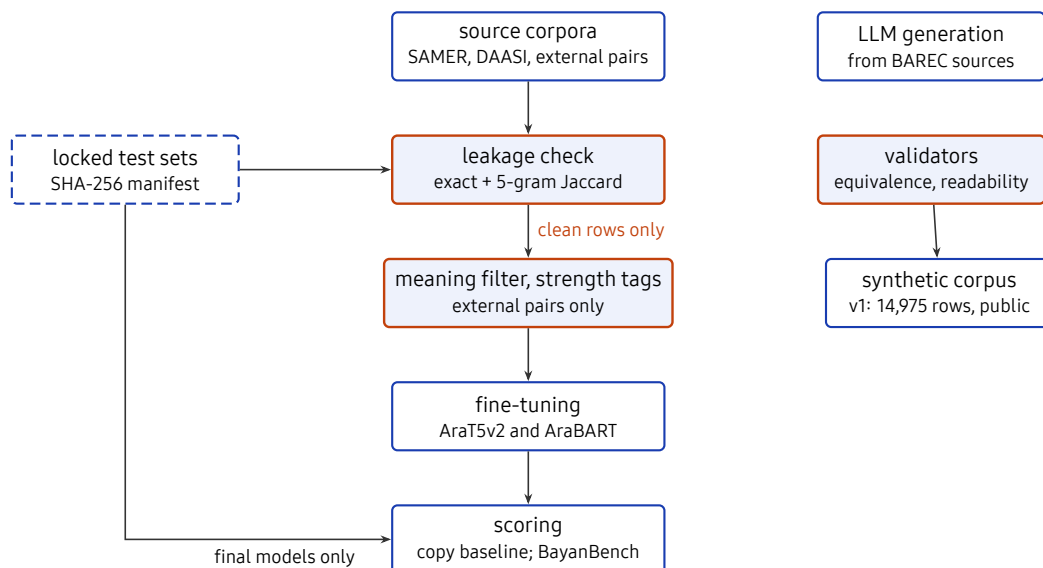


Figure 3. Data and evaluation workflow. Orange boxes are the gates that can reject data. The generated corpus (right) is a public artefact that feeds BayanSimplify-v0.3.

2.4. System Design

Figure 4 shows the system. The Android app registers an activity for `android.intent.action.PROCESS_TEXT`, so تبسيط appears in the selection menu of any app, and a translucent activity draws the result over the host app; an optional floating panel keeps reading while the reader returns to the page.

Inference. Each simplifier ships as an int8 ONNX bundle (encoder, decoder with a key–value cache, and tokenizer), downloaded once, verified by SHA-256 and run by ONNX Runtime with a decoding loop written in Kotlin. The fast model and BayanSimplify-v0.2 decode greedily and stream each token as it is produced. BayanSimplify-v0.3, the large model, decodes with four beams (the setting *More faithful, slower*, on by default; #61, #62, #66) and shows each sentence when it is final: a port of transformers’ beam search that gives transformers’ exact tokens on 120 of 120 BayanBench sentences. The app strips tashkeel, splits the selection into sentences and decodes them one by one; sentences under six words (headings, labels) pass through unchanged.

The app’s text step. Before the model, the selection is cleaned (PDF line breaks, presentation forms, list bullets), digits are mapped to Western digits, and scripture and set poetry are detected and *never sent to the model*. After it, a sentence keeps its source wording when the rewrite keeps less than 35% of its content words, drops or changes a number or a Latin-script word, or changes the count of negations, limits (*at least, only*) or condition words. On the BayanBench dev core items these guards raise numbers kept from 72% to 100% and protected text and cut-off endings to 100%, remove every flagged negation and limit change, and raise meaning kept by about 3.5 points; they fall back on 3–7% of sentences. The original is always one tap away.

Read-aloud and diacritization. Voices run on the device through sherpa-onnx; the default, Nabra-7M-Distill, is an 8.7 MB int8 Arabic voice, and the spoken word is highlighted. The diacritizer, Libtashkeel (Table 18), is a 4.8 MB character model bundled with the app. With *Show tashkeel* on (off by default, since full vowels can crowd a line), each finished sentence gets its short vowels after the guards; the Kotlin port gives the Python package’s exact output on 305 BayanBench texts and never changes a letter.

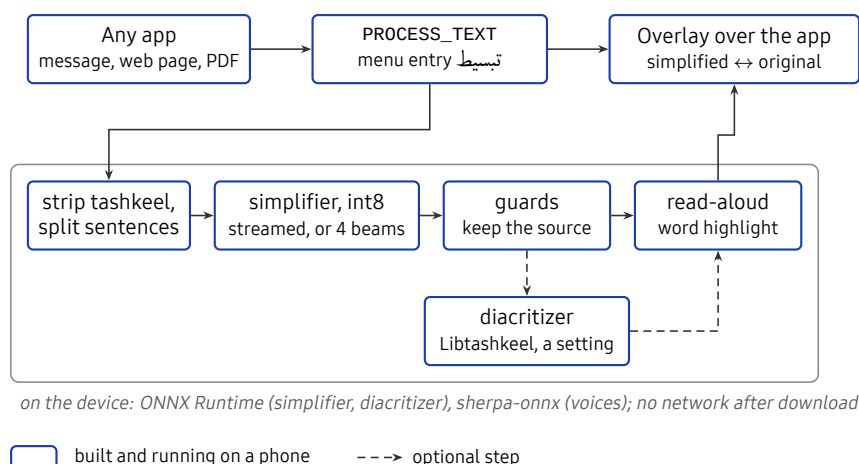


Figure 4. System architecture of the build tested on the phone. The app is entered from the host app’s selection menu and answers with an overlay on top of it.

3. Results

3.1. Data Preprocessing

SAMER’s aligned levels were converted to source–target pairs (level 5 → level 3), and one converter turns each locked test file into the scoring format, keeping every row and checking the count. For leakage checks and alignment, diacritics and punctuation are removed and the alef, ؤ/ة and ي/ي variants unified; training text keeps its spelling apart from tashkeel and tatweel. Figure 5 shows the pipeline that produced corpus v1: protected text and short headings are copied unchanged, and a rewrite must pass four gates.

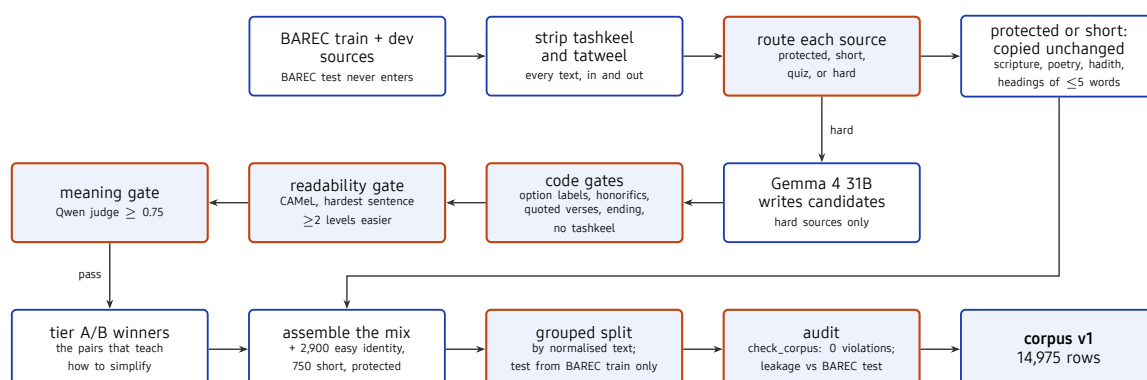


Figure 5. Data cleaning pipeline for corpus v1. Orange boxes can reject data. Counts are from `assemble_corpus_v1.py` and `check_corpus.py`.

Leakage. Every training file was checked against both locked test sets and matching rows removed until none matched; BAREC alone repeats 311 sentences between its own train and

test splits, and 9.3% of a public Arabic simplification corpus overlaps the SAMER and BAREC test sets.

Meaning filter. Applied to the Baseet candidates, the filter keeps 6,888 pairs. Its purpose shows at the strongest level: the 5th percentiles of its level-1 targets are 0.44 LaBSE similarity and 26% of the source’s length, and 10% of them drop a number, against 0.646, 67% and none for DAASI’s human rewrites.

3.2. Exploratory Data Analysis (EDA)

Table 7. Corpus statistics, computed on the files we hold. Identity = target equals source after removing diacritics; length ratio = median of target words over source words.

Corpus	Rows	Median words	≤ 3 words	Diacritized	Identity	Length ratio
SAMER train	14,343	7	16.4%	0.0%	42.1%	1.00
SAMER test (locked)	3,277	6	23.6%	0.0%	48.8%	1.00
BAREC test (locked)	7,286	10	16.9%	53.8%	—	—
DAASI	1,509	12	8.4%	32.3%	0.0%	1.00

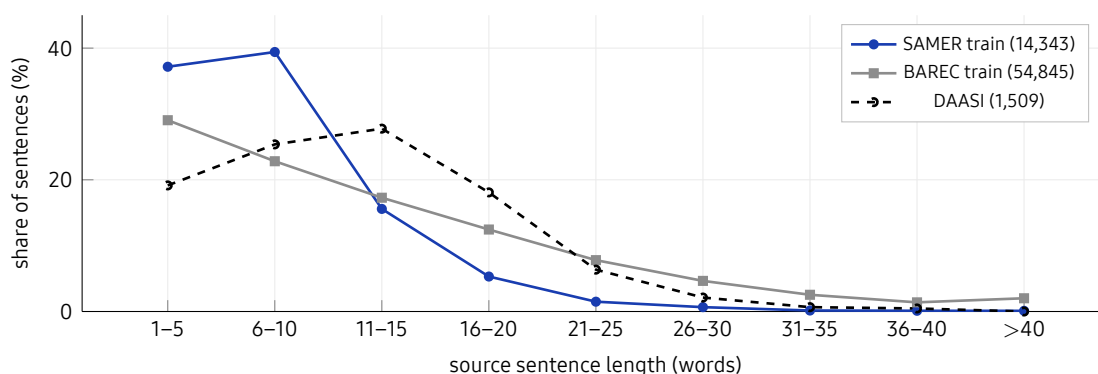


Figure 6. Source sentence length by corpus. SAMER is short literary text; BAREC reaches the long sentences readers struggle with.

Four properties of the corpora (Table 7, Figure 6) explain the behaviour of the models:

- **SAMER carries no diacritics**, while 54% of BAREC test does, so a model trained only on SAMER has never seen a diacritized input.
- **SAMER simplifies words, not sentences**. Its median length ratio is exactly 1.00: the editors substituted vocabulary and kept structure.
- **No corpus teaches splitting**. A sentence is added in 0% of SAMER pairs, 2–4% of Baseet’s and 8% of DAASI’s, so existing data cannot teach the restructuring the reading research supports most [6].
- **Half of SAMER needs no change**. 42–49% of pairs are identical, and SAMER splits by chapter, so its test set is in-domain: both make SARI on SAMER read optimistically.

3.3. Modeling

RQ1: the reference-based view. Reference-based scores use SARI [18] beside BLEU, BERTScore [19] and the fall in predicted BAREC level, always paired with the copy base-

line and split into rows a human changed and rows a human left unchanged. Because half of SAMER needs no change, copying scores SARI 77.50 on the full test set but only 56.06 on the 1,678 rows a human changed, and the second number is the bar a model must clear (Table 8, Figure 7).

Table 8. BayanSimplify-v0.1 on test data with human references, scored once with score.py. SARI is split by whether the human reference changed the source (DAASI has no unchanged pairs). BERTScore compares output with source; readability drop is the fall in predicted BAREC level (positive = easier). DAASI sources and references have tashkeel stripped.

System	SARI all	SARI changed	SARI un- changed	BLEU	BERT- Score	Read- ability drop	Copy rate
SAMER test (3,277 rows, locked, in domain)							
Copy the input	77.50	56.06	100.00	72.03	1.00	0.00	100.0%
v0.1, greedy	75.88	61.83	90.62	71.68	0.98	0.08	65.3%
v0.1, beam 4	76.29	61.95	91.34	72.22	0.98	0.07	69.4%
DAASI held-out (350 pairs, government and insurance text, out of domain)							
Copy the input	45.05	45.05	—	25.25	1.00	0.00	100.0%
v0.1, greedy	43.87	43.87	—	23.66	0.99	-0.03	77.7%

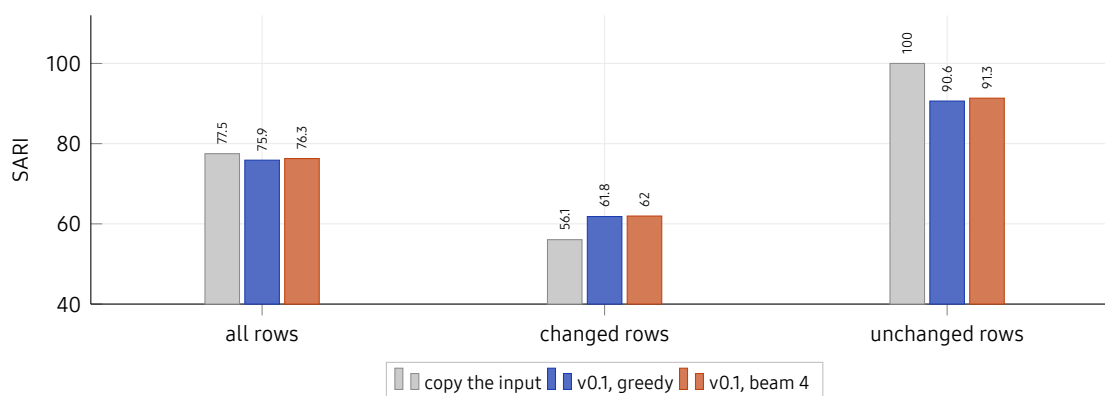


Figure 7. SARI on the locked SAMER test set by reference bucket. BayanSimplify-v0.1 clears the copy baseline where a human simplified and loses to it where a human left the sentence alone.

BayanSimplify-v0.1 simplifies where it should and over-edits where it should not. On the 1,678 test rows a human simplified, greedy decoding scores 61.83 against the copy baseline's 56.06 (+5.77 SARI). On the 1,599 rows a human left alone it scores 90.62 against a perfect 100, because it edits 13.6% of sentences that needed no change. The two effects net to -1.62 overall, so the headline number would report a working model as worse than doing nothing.

What BayanSimplify-v0.1 learned. It returns 65.3% of test rows unchanged where the editors left 48.8%, and when it edits, 75.4% of its edits change one word (editors: 49.5%): it substitutes vocabulary and never restructures (Figure 8a). On BAREC's 3,918 test sentences that carry tash-keel, raw input keeps only 63.7% of the source words and 31.9% of outputs lose more than half the sentence; with tashkeel stripped, 94.6% and 1.8% (Figure 8b). On DAASI's administrative text it scores 43.87 against a copy baseline of 45.05 (-1.19). Some of its substitutions reverse the meaning: عزلة (isolation) became استعباد (enslavement). SARI is blind to this class of error, which is why BayanBench scores meaning separately.

v0.2 training. Both v0.2 runs clear BayanSimplify-v0.1's selection score: SARI 66.58 (AraT5v2)

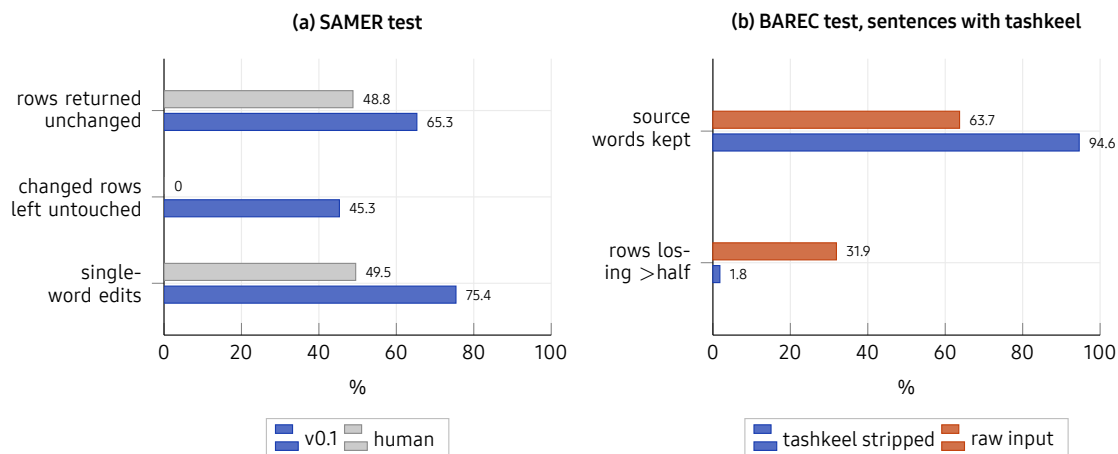


Figure 8. BayanSimplify-v0.1 behaviour. (a) SAMER test, greedy, against the editors' references. (b) BAREC test, the 3,918 sentences that carry tashkeel, greedy, with and without tashkeel stripped.

and 64.71 (AraBART) on the 600 changed rows of the selection sample, where BayanSimplify-v0.1 scored 63.66 on SAMER dev's changed rows. Copying falls from 80–91% of rows early in training to about 51%, against 33.3% for the editors. AraBART trains in 2,600 s against 6,783 s and generates 292.8 sentences per second against AraT5v2's 80.8 (Table 9, Figure 9).

Table 9. The v0.2 runs, one T4 GPU each. SARI and copy rate on the 900-row selection sample (600 changed, 300 unchanged SAMER dev rows, tag [s0]). Speed: sentences per second on SAMER test sources, fp16, batch 64, greedy.

	AraT5v2	AraBART
Parameters	368M	139M
Training time	6,783 s	2,600 s
Best checkpoint (step of 14,210)	14,200	12,780
SARI, 600 changed rows	66.58	64.71
SARI, all 900 rows	75.64	73.62
Copy rate, 900 rows (editors: 33.3%)	51.0%	51.4%
Generation speed (sentences/s)	80.8 [†]	292.8

[†] measured on BayanSimplify-v0.1, the same architecture and size, in the same run.

The scorer against people. On the 80 outputs given to every rater, the raters agreed moderately on meaning (Krippendorff's $\alpha = 0.51$) and weakly on Arabic ($\alpha = 0.17$) and ease ($\alpha = 0.19$), so meaning is the one human judgement firm enough to check an automatic scorer against. The scorer's P(same meaning) separates the outputs the raters kept from the rest with an AUC of 0.85 [0.78–0.91] (Figure 10). As a pass/fail rule at 0.5 it catches 42 of the 51 outputs the raters judged changed, including all 11 major changes, and fails 53 of the 194 they kept: it is stricter than people, so “meaning kept” is a conservative rate. A cut-off of 0.3 would have failed seven fewer kept outputs and caught the same losses; we kept the 0.5 set before the ratings rather than tune it on the sample used to check it. Per system (Table 10), the raters kept the meaning in 94% of the int8 AraT5v2 bundle's rewrites against 71% for each AraBART system, and the scorer places every AraT5v2 system above every AraBART one too. The reader with dyslexia (Table 11) found 17 of 30 rewrites easier, 9 no different and 4 harder, and agreed with the team's majority on 67% of shared outputs, about as often as two team raters agreed on ease; the int8 AraT5v2 bundle, the most cautious system, was the only one never found easier.

RQ2: the v0.2 models on BayanBench. Each system was run once: the v0.2 models in the

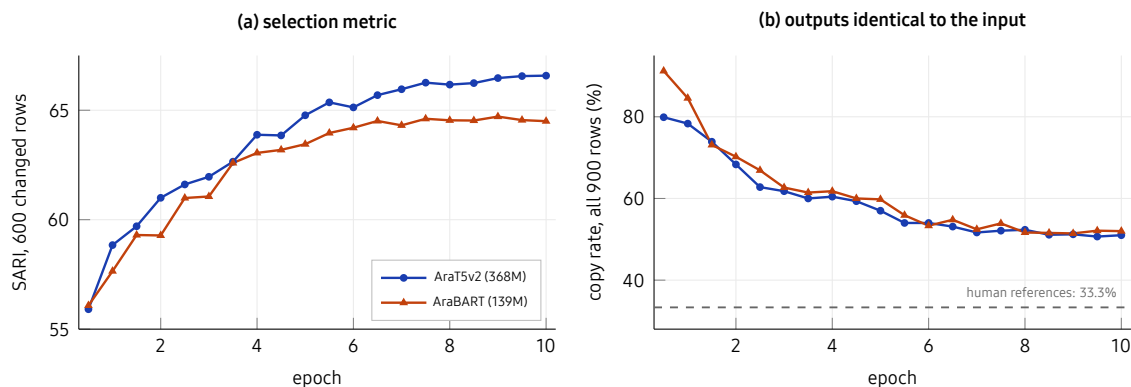


Figure 9. The v0.2 training, from the committed notebooks: (a) the selection metric at each of the 20 evaluations; (b) the share of outputs identical to the input, with the editors' rate as a dashed line.

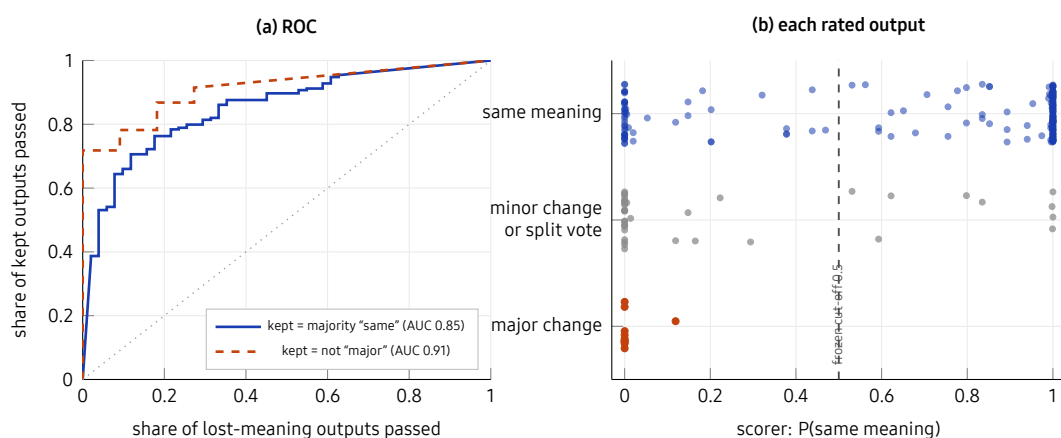


Figure 10. The meaning scorer against the raters, on 245 rated dev outputs. (a) ROC curves for two definitions of a kept meaning; the dotted diagonal is chance. (b) Every rated output by the raters' majority answer (194 same, 40 minor or split, 11 major) against the scorer's P(same); rows are jittered vertically for legibility.

medium setting ([S2]), the two int8 bundles exactly as the app calls them, and copying the input (Table 12, Figure 11).

- F1 Meaning and simplification trade off.** The medium setting cuts 6.3 (AraT5v2) and 6.7 (AraBART) words from the longest clause and keeps the meaning in 59.5% and 40.1% of outputs; the int8 AraT5v2 bundle cuts 1.9 and keeps 78.6%. No v0.2 system simplifies as much as the tagged setting while keeping meaning as often as the shipped bundle.
- F2 AraT5v2 keeps meaning better** than AraBART at every setting (19.4 points with the [S2] tag, 28.1 as an int8 bundle), as the raters also found.
- F3 Simplification is mostly structural.** Reading level falls by at most 0.40 levels, where a clause loses up to 6.7 words.
- F4 Restraint is weak.** Easy text comes back unchanged in only 11.7–48.1% of C items and protected text keeps its letters in 4.4–47.8%; cut-off selections are often completed with invented words (Table 15). The app's text step now handles protected text, numbers and cut-off endings in code, and BayanSimplify-v0.3 improves restraint.
- F5 The deployed format buys meaning with ease.** The app's bundles send no strength tag, which raises meaning kept and cuts fewer words; int8 moves AraT5v2 further toward copying, which makes the shipped AraT5v2 bundle the safest v0.2 system and the least simplifying one (Table 13). Int8 rewords most of AraT5v2's outputs: only 234 of 731 dev outputs

Table 10. Each system as the raters and the scorer saw the same outputs: the 35 changed outputs per system drawn evenly (copies of the input were not rated). Rates in %, a strict majority of the task’s raters; 95% intervals from a bootstrap over documents. *Scorer:* $P(\text{same}) \geq 0.5$ on the same outputs.

System	Same meaning: raters	Same meaning: scorer	Easier: raters	Correct Arabic: raters
v0.2, int8 app	94 [85–100]	80 [67–92]	49 [30–67]	91 [82–100]
v0.2, no tag	89 [78–97]	83 [71–94]	54 [36–72]	89 [77–97]
v0.2, [s2]	77 [62–91]	63 [46–78]	49 [34–65]	97 [90–100]
v0.2-Fast, int8 app	71 [56–86]	51 [31–72]	54 [38–70]	94 [86–100]
v0.2-Fast, no tag	71 [52–88]	54 [38–71]	51 [33–68]	91 [82–100]
v0.2-Fast, [s2]	71 [57–86]	49 [34–63]	57 [43–72]	97 [92–100]

Table 11. The reader with dyslexia: “was it easier for you?” on 30 outputs (five per system), against the original.

System	Easier	Same	Harder
v0.2, int8 app	0	3	2
v0.2, no tag	4	0	1
v0.2, [s2]	3	2	0
v0.2-Fast, int8 app	3	2	0
v0.2-Fast, no tag	5	0	0
v0.2-Fast, [s2]	2	2	1
All	17	9	4

match the float model’s.

Across item sets. Meaning is kept at similar rates on text published after every corpus we use (outside) as on core (Table 14), so the results do not rest on BAREC’s genres; restraint is weak everywhere.

Error analysis. Table 15 shows test outputs of AraT5v2 in the medium setting. The first is the intended behaviour; the second passes every code check and reverses a quantity, which is why meaning needs a scorer. In a Qur’anic verse (109:4) the model dropped the negation ﻻ, the most serious error we observed and the reason protected text never reaches the model.

BayanSimplify-v0.3: our best model. As float models, BayanSimplify-v0.3 leaves easy text unchanged in 55% of cases against BayanSimplify-v0.2’s 14% and keeps protected text intact in 97.8% against 4.4%. It was gated as it would ship: its int8 bundle through the app’s text step, against BayanSimplify-v0.2 int8 through the same step, under a rule set before the run (meaning kept not significantly lower, the longest clause cut more, no check worse). With greedy decoding it passed on dev but lost 6 points of meaning kept on test [–10, –1].

Beam search makes BayanSimplify-v0.3 the best model we have measured. #61 (Khengari) showed on BayanSimplify-v0.3’s int8 bundle that four beams keep the meaning 4–8 points more often than greedy decoding, and the app gained beam search with transformers’ exact tokens (#62, #66). Run with four beams by the app’s own engine, through its text step, with every core item scored, BayanSimplify-v0.3 outperforms every model in Table 12: it keeps the meaning in 82% of test outputs, against at most 78.6%, and cuts 5.2 words from the longest clause.

Table 12. The v0.2 models on BayanBench v2, **test** split, *core* items (749 of 1250 items; 197 documents in the split), each system run once. Meaning: the open scorer, Gemma 4 31B; a copy of the input keeps its meaning by definition. Simpler: mean change from selection to output, over the items where there is something to simplify. Rates in %, means with their sign; 95% intervals from a bootstrap over whole documents in brackets. The two [S2] columns send the medium strength tag the model was trained with; the two *int8 app* columns are the quantised bundles the app downloads, which send no tag. All outputs pass through the app's guards. Bold: the best model on each row. *n* = items the measure applies to.

Measure	Copy the input	v0.2, [S2]	v0.2-Fast, [S2]	v0.2, int8 app	v0.2-Fast, int8 app	<i>n</i>
Meaning						
Meaning kept (P(same) $\geq$ 0.5, numbers kept), %	100.0	59.5 [55–64]	40.1 [36–44]	78.6 [75–82]	50.5 [46–55]	659
Mean P(same meaning)	1.00	0.59 [0.55, 0.64]	0.40 [0.37, 0.45]	0.79 [0.75, 0.82]	0.51 [0.47, 0.56]	659
Simpler (mean change)						
Longest clause: words cut (clause over 15)	+0.0	+6.3 [+5.0, +7.9]	+6.7 [+5.4, +8.3]	+1.9 [+1.0, +2.9]	+4.8 [+3.5, +6.1]	212
Reading level: levels lower (level 10+)	+0.00	+0.40 [+0.31, +0.49]	+0.38 [+0.30, +0.47]	+0.14 [+0.08, +0.20]	+0.28 [+0.19, +0.38]	543
Hard words removed (items with any)	+0.00	+0.61 [+0.46, +0.81]	+0.68 [+0.54, +0.85]	+0.31 [+0.22, +0.44]	+0.59 [+0.46, +0.75]	328
Checks, %						
Every number kept	100.0	82.7 [65–95]	78.7 [63–90]	84.0 [70–94]	76.0 [59–88]	75
No house-rule break	100.0	99.8 [99–100]	99.7 [99–100]	100.0	100.0	659
Behaviour, %						
Easy text left unchanged (C)	100.0	14.3 [7–22]	11.7 [5–19]	48.1 [36–61]	24.7 [15–35]	77
Cut-off selection keeps its ending (E1)	100.0	36.5 [25–47]	38.8 [27–50]	38.8 [28–49]	45.9 [34–58]	85
Protected text: letters unchanged (F)	100.0	4.4 [0–10]	4.4 [0–11]	47.8 [30–70]	24.4 [16–36]	90
Flags (not failures), %						
Negation count changed	0.0	23.1 [17–30]	24.1 [17–32]	7.1 [4–11]	18.4 [13–25]	139

Even against BayanSimplify-v0.2 behind the same newest checks it is no worse on meaning (–3 points on test [–7, +1], +1 on dev [–4, +7]) and simplifies far more: 3.4 more words cut on test [+2.4, +4.4], easy text left unchanged 18 points more often, and every number kept. Khengari's benchmark report of 35 systems (#57, #58) reaches the same verdict: against BayanSimplify-v0.2 in the app it keeps every number (100% against 84%), changes no negation, limit or condition word, leaves easy text unchanged in 74% of cases against 48%, and raises meaning kept by 4 points [+0, +8].¹ The headroom it shows is on the phone: int8 keeps about half of the float model's drop in reading level, and BayanSimplify-v0.3 leaves 47% of items unchanged (the app's earlier model: 19%), rewriting less often but harder.

People agree. A second blind round (4–5 October) rated the app's three models as the reader gets them, on 18 BayanBench test sentences that BayanSimplify-v0.3 never trained on, with

¹His report puts the system at 83% meaning kept and 5.4 words cut on test; this section's run gives 82% and 5.2. Both decode the same int8 bundle with four beams; his applies the safety net to transformers' outputs on another CPU, where int8 rounding differs slightly, while ours runs the app's own decoder and its full text step. The comparisons they support point the same way.

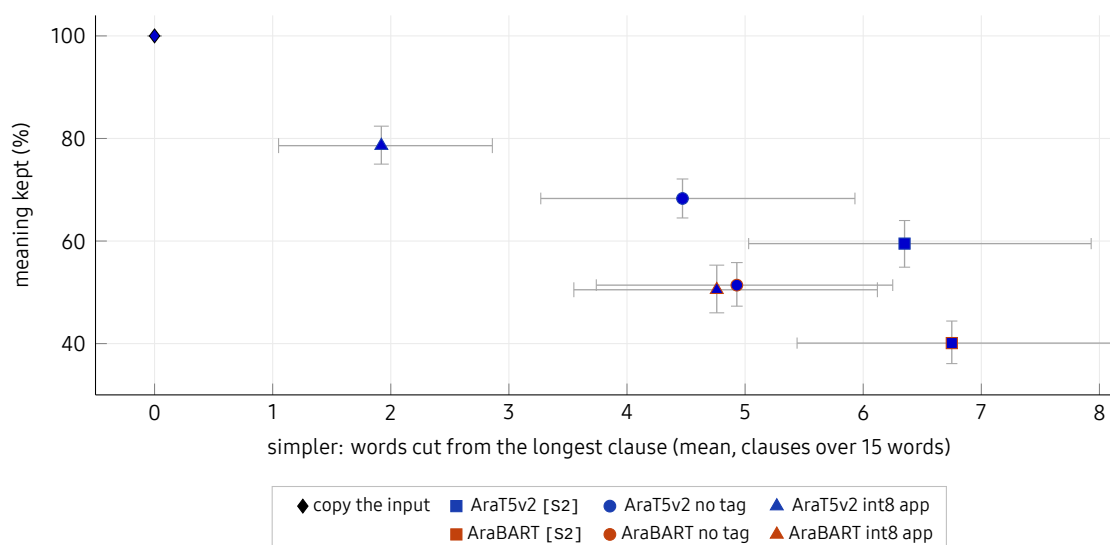


Figure 11. Meaning kept against simplification, test split, core items, with 95% document-bootstrap intervals. Copying the input sits at the top left by definition. Colour: the model; shape: the setting.

Table 13. Paired effects on the test split (core items): one change at a time, same items. Differences in points (rates) or words (clause), with 95% document-bootstrap intervals; * = the interval excludes zero.

One change	Meaning kept (points)	Words cut from the longest clause	Easy text unchanged (points)
AraT5v2: drop the [S2] tag	+8.8* [+5.9, +12.0]	-1.9* [-2.4, -1.4]	+13.0* [+4.8, +21.4]
AraBART: drop the [S2] tag	+11.4* [+7.0, +15.5]	-1.8* [-2.5, -1.2]	+11.7* [+3.4, +19.1]
AraT5v2: PyTorch → int8 app	+10.3* [+7.1, +13.5]	-2.6* [-3.3, -1.9]	+20.8* [+11.9, +31.1]
AraBART: PyTorch → int8 app	-0.9 [-3.0, +1.1]	-0.2 [-0.5, +0.2]	+1.3 [-3.2, +5.9]
[S2]: AraBART → AraT5v2	+19.4* [+14.9, +23.5]	-0.4 [-0.9, +0.2]	+2.6 [-6.8, +12.5]
No tag: AraBART → AraT5v2	+16.8* [+12.1, +21.6]	-0.5 [-1.1, +0.2]	+3.9 [-8.6, +16.7]
int8 app: AraBART → AraT5v2	+28.1* [+23.3, +32.6]	-2.8* [-3.7, -2.0]	+23.4* [+11.3, +36.9]

a Latin square giving each of the 4 team raters every sentence once. BayanSimplify-v0.3 kept the meaning as often as any model (96% of ratings; BayanSimplify-v0.2 96%, the fast model 92%) and was found easier most often: on the outputs it changed, 86% against 60% and 45%, with none rated harder. Every team vote between BayanSimplify-v0.3 and BayanSimplify-v0.2 chose BayanSimplify-v0.3 (8 of 8), and BayanSimplify-v0.3 won 5 of 8 votes against the fast model. The reader with dyslexia, who rated all 24 tasks, chose BayanSimplify-v0.3 in every comparison that included it and found 4 of its 6 single outputs easier and none harder. The round is small, so it confirms the direction of the benchmark rather than measuring a size. BayanSimplify-v0.3 with four beams is therefore the app's large model (5 October); a reader who prefers word-by-word streaming can turn the setting off.

Table 14. The four item sets on the test split, reported side by side and never averaged; number of items in brackets. *Core*: BAREC test documents; *held-out*: news and legal text; *outside*: web text published after every corpus we use; *written*: medicine-leaflet sentences written for the benchmark.

System and measure	Core	Held-out	Outside	Written
v0.2, [s2]				
Meaning kept, %	59.5 ⁽⁶⁵⁹⁾	64.6 ⁽²⁷⁴⁾	60.8 ⁽²⁰⁴⁾	65.2 ⁽²³⁾
Longest clause: words cut	+6.3 ⁽²¹²⁾	+5.8 ⁽⁵⁸⁾	+3.5 ⁽⁶⁵⁾	+2.0 ⁽¹⁾
Easy text left unchanged, %	14.3 ⁽⁷⁷⁾	6.5 ⁽⁹²⁾	6.1 ⁽³³⁾	—
v0.2-Fast, [s2]				
Meaning kept, %	40.1 ⁽⁶⁵⁹⁾	49.3 ⁽²⁷⁴⁾	37.7 ⁽²⁰⁴⁾	52.2 ⁽²³⁾
Longest clause: words cut	+6.7 ⁽²¹²⁾	+5.5 ⁽⁵⁸⁾	+4.9 ⁽⁶⁵⁾	+4.0 ⁽¹⁾
Easy text left unchanged, %	11.7 ⁽⁷⁷⁾	7.6 ⁽⁹²⁾	21.2 ⁽³³⁾	—
v0.2, int8 app				
Meaning kept, %	78.6 ⁽⁶⁵⁹⁾	77.0 ⁽²⁷⁴⁾	75.5 ⁽²⁰⁴⁾	82.6 ⁽²³⁾
Longest clause: words cut	+1.9 ⁽²¹²⁾	+2.3 ⁽⁵⁸⁾	+2.1 ⁽⁶⁵⁾	-1.0 ⁽¹⁾
Easy text left unchanged, %	48.1 ⁽⁷⁷⁾	22.8 ⁽⁹²⁾	27.3 ⁽³³⁾	—
v0.2-Fast, int8 app				
Meaning kept, %	50.5 ⁽⁶⁵⁹⁾	56.9 ⁽²⁷⁴⁾	46.6 ⁽²⁰⁴⁾	69.6 ⁽²³⁾
Longest clause: words cut	+4.8 ⁽²¹²⁾	+3.9 ⁽⁵⁸⁾	+4.2 ⁽⁶⁵⁾	+0.0 ⁽¹⁾
Easy text left unchanged, %	24.7 ⁽⁷⁷⁾	18.5 ⁽⁹²⁾	12.1 ⁽³³⁾	—

Table 15. Test outputs of AraT5v2 ([s2]), from the BayanBench test split. Glosses are ours.

Track	Source	Output	What happened
B	فالمهم الصفح والتسامح وذلك للتخفيف على الصحة النفسية للفرد ولانتشار المحبة والتعاون بين أفراد المجتمع بشكل عام.	فالمهم الصفح والتسامح، لأن ذلك يساعد على تحسين الصحة النفسية للفرد، ويساعد على انتشار المحبة والتعاون بين أفراد المجتمع بشكل عام.	Split into two clauses joined by a stated cause; meaning kept.
B	منذ قرنين على الأكثر بدأ الإنسان يدخل دائرة حلزونية متصاعدة متفاقمة بين الطلب والاستجابة للطلب ...	منذ قرنين على الأقل، بدأ الإنسان يدخل دائرة حلزونية متصاعدة بين الطلب والاستجابة له ...	“At most two centuries” became “at least”: a reversal every code check passes.
C	يعتبر منصب الرئيس شاغرا في أي من الحالات التالية:	يعتبر منصب الرئيس شاغرا في أي حالة:	Easy legal text edited; “in any of the following cases” lost its list.
E1	الشكل المنطقي الأساسي في علم المنطق، ويتم تحديد صحة المفاهيم من خلال صحة المفاهيم.	الشكل المنطقي الأساسي في علم المنطق، ويتم تحديد صحة المفاهيم من خلال اختبار صحة المفاهيم.	A cut-off selection completed with invented words and a full stop.
A2	... بنتيجة (١/٣) في المباراة ...	... بنتيجة (١) في المباراة ...	Persian-range digits dropped.

3.4. User Interface

Figure 12 shows the interface on the phone. Selecting Arabic text in any app offers تبسيط (Simplify) among the selection actions; choosing it opens the result over the app. One toggle switches between *Simplified* and *Original*; below it are read-aloud, copy and share, and the current word is highlighted while Bayan reads. The panel can be minimised to a bar over the page that keeps reading, with play and stop also in the phone’s media controls. The app follows Android’s Material 3 design, takes its colours from the phone’s Material You palette, and sets Arabic right to left without justification, which in Arabic stretches words. Settings hold the reader’s typeface, size, line height and word spacing, the background, tashkeel, the voice and its speed, and the model, with a sample sentence that shows each change as it is made.

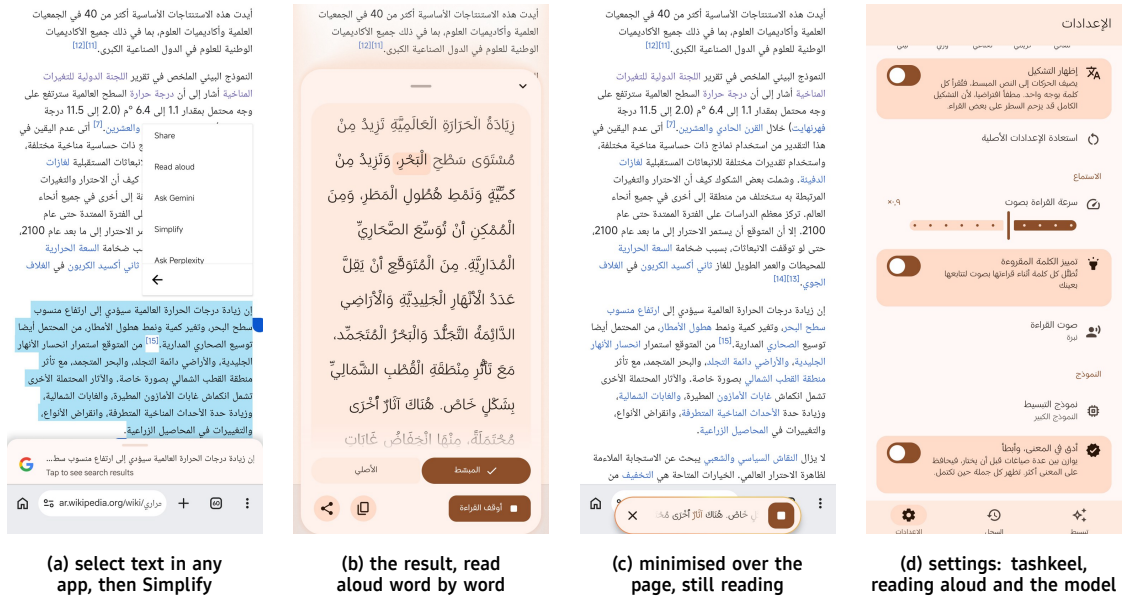


Figure 12. Bayan 2.1.0 on the phone (Xiaomi Mi 11X), over an Arabic Wikipedia page, with BayanSimplify-v0.3 (the large model, four beams), tashkeel shown and a reader’s own size and spacing; the phone’s palette is seeded with the ochre of Bayan’s logo. (a) The selection menu. (b) The result sheet, reading aloud with the current word highlighted. (c) Minimised over the page while reading continues. (d) Settings: *Show tashkeel*, reading aloud, and the model with *More faithful, slower*.

3.5. Testing and Improvements

Testing. The first plugin shell shipped 41 unit and Robolectric tests; the app unit-tests its text pipeline, and its download pipeline checks every file by SHA-256. The leakage checker is tested against a planted duplicate. The benchmark package has 12 tests, and its prediction command reproduces our v0.2 outputs on all 731 dev items, as an independent rerun by a teammate confirmed. Running the pipeline on real data exposed four harness faults: every BAREC row scored as “unchanged”; BayanSimplify-v0.1’s training prefix missing from the shared prediction script (twelve SARI points); length-grouped batching shuffling the evaluation set; and garbled outputs under transformers 5. Each now has a guard.

Improvements. Each measured problem produced a change (Table 16).

RQ3: on the phone. Both architectures were exported to ONNX with the tokenizer inside the bundle and quantised to int8 (Table 17). The AraBART bundle, the default, fits the 250 MB budget; the AraT5v2 bundles (BayanSimplify-v0.3 and BayanSimplify-v0.2) are optional downloads of 471 MB, 46% of it two untied embedding matrices that we kept whole rather than trade qual-

Table 16. What the measurements changed.

Finding	Change
Copying scores SARI 77.5 on SAMER test	Copy baseline and changed/unchanged split reported with every score
BayanSimplify-v0.1 collapses on diacritized input	Tashkeel stripped before the simplifier, in training and in the app
BayanSimplify-v0.1 chosen on SARI noise over a copy-heavy dev set	The v0.2 models select on changed-row SARI only
SAMER teaches word substitution	DAASI and meaning-filtered Baseet pairs added, each under its own tag
LLM-written Baseet targets often summarise	Meaning filter calibrated on human rewrites
Prediction script omitted the training prefix	Prefix a required setting of every model and bundle
Length grouping shuffled the evaluation set	Grouping off; alignment check stops the run
SARI is blind to meaning errors	BayanBench: meaning scored separately, by an open scorer checked against six raters
Some BayanBench v1 measures did not track human labels	Dropped; v2 keeps only validated measures
Under transformers 5 the models' outputs changed silently	Prediction refuses to run under version 5
The v0.2 models over-edit easy and protected text	Corpus v1 and BayanSimplify-v0.3; scripture and set poetry never reach the model
The model drops numbers and flips negations	The app keeps the source sentence when a rewrite loses a number, a negation or a limit
The model invents text around headings and fragments	Sentences under six words pass through unchanged in the app
BayanSimplify-v0.3 greedy lost meaning on test	Beam search in the app; four beams by default

ity for size. On a Xiaomi Mi 11X (Snapdragon 870, Android 13, 6 GB RAM), with six cold starts each, the reader waits 82–172 ms for AraBART's first token and 177–318 ms for AraT5v2's. With four beams BayanSimplify-v0.3 gives up streaming and takes 1.19 s per sentence (median over 120 BayanBench dev sentences), against 0.41 s greedy. Quantisation is not neutral for AraT5v2: int8 changed 62–74% of its outputs against the float model, though not the rate at which meaning is kept.

Table 17. On-device cost. Laptop: 4 CPU threads, median and 90th percentile per BayanBench test sentence. Phone: Xiaomi Mi 11X, from the app's logs over six cold starts each on the same two-sentence paragraph.

	AraBART, int8	AraT5v2, int8
Download (encoder + decoder + tokenizer)	222 MB (212 MiB)	471 MB (449 MiB)
Laptop CPU, per sentence (median / p90)	0.10 / 0.24 s	0.20 / 0.43 s
Phone: model load, cold	1.0–1.7 s	2.6–7.3 s
Phone: first token, first sentence	82–172 ms	177–318 ms
Phone: first token, later sentences	28–30 ms	55–85 ms
Phone: BayanSimplify-v0.3 per sentence, greedy / 4 beams (median, 120 dev sentences)	—	0.41 / 1.19 s
Read-aloud voice (Nabra-7M-Distill, int8)	8.7 MB	

Diacritization. Four systems were benchmarked on the 743-sentence Tashkeela test fixture (Table 18). Libtashkeel was selected: CATT ties it on accuracy but is 16 times larger and non-commercial, and CAMEL Tools changed the base letters of 32 of 100 sentences. A re-run before shipping reproduced the table exactly, and newer models did not displace it (the rawi models

rewrite hamza seats in 97–106 sentences; Tashkeel-v4 is for research use only).

Table 18. Diacritization benchmark. DER = diacritic error rate; base-letter corruption measured on 100 sentences.

System	DER	DER, no case	WER	Base letters changed	Size	Licence
CAMeL Tools MLE [39]	16.57%	13.97%	56.73%	32%	129 MB	MIT/GPL
CATT [40]	8.44%	6.92%	29.40%	0%	77.9 MB	CC BY-NC
Mishkal [41]	12.74%	10.12%	39.33%	0%	4.3 MB	GPLv3
Libtashkeel [42]	8.43%	6.91%	29.57%	0%	4.8 MB	MIT

4. Projected Impact

4.1. Accomplishments and Benefits

Table 19. Objectives from subsection 1.2, against the evidence.

Objective	Evidence	Outcome
O1 Beat copying on changed rows; lower linguistic load without losing meaning	BayanSimplify-v0.1: 61.83 vs 56.06 on SAMER test (+5.77). The v0.2 models lower clause length and hard words, but keep the meaning in 40.1–78.6% of outputs. BayanSimplify-v0.3 with four beams outperforms them: 82% meaning kept on test, 5.2 words cut from the longest clause	partly met
O2 Diacritize without changing letters	Libtashkeel in the app (a setting): DER 6.91% without case endings, no letter changed in 743 + 300 sentences, 4.8 MB	met
O3 Run on a phone; default within 250 MB	Both bundles run on a Snapdragon 870 phone, first token in 28–318 ms; the default AraBART int8 is 222 MB; the optional AraT5v2 int8 bundles (BayanSimplify-v0.3, the large model, and BayanSimplify-v0.2) are 471 MB	met
O4 Evidence from readers	A reader with dyslexia rated rewrites for ease, with informed consent, and chose BayanSimplify-v0.3 in every comparison that included it. No one with dyslexia has used the app, and there is no reading-time or comprehension study	started

RQ1: no. On copy-heavy test sets SARI rewards inaction and is blind to reversals of meaning; an open meaning scorer, by contrast, agrees with human raters (AUC 0.85) and can be rerun by anyone. **RQ2:** partly. Every v0.2 setting traded meaning for simplification; BayanSimplify-v0.3 with four beams moves the trade-off, keeping the meaning more often than any earlier model while simplifying more than BayanSimplify-v0.2 in the app. **RQ3:** yes. Streamed output reaches the reader in under a third of a second with either bundle, BayanSimplify-v0.3 with four beams shows a sentence in about 1.2 s, and the default fits the 250 MB budget; the large model needs 471 MB.

Benefits. For readers: a simpler version where and when the difficulty occurs, offline and private, with the original one tap away. For schools: a tool that runs on the phones students already own. For the field: a documented case of test-set contamination in a public Arabic simplification corpus, an evaluation protocol that exposes copy-heavy scores, and a benchmark, a meaning judge and four public models.

4.2. Future Improvements

The central next step is to simplify as much as the boldest setting while keeping meaning as well as the safest. The meaning scorer, now checked against people, can rank candidate rewrites and select checkpoints, so that training is steered by the behaviours readers need rather than by SARI. Faster beam decoding would let BayanSimplify-v0.3's quality arrive word by word; pruning the AraT5v2 vocabulary (46% of its parameters sit in two embedding matrices) could bring its 471 MB bundle to about 223 MB, if the full evaluation shows no loss; and sentence splitting needs training data that teaches it. Above all, a reading study with more readers with dyslexia, measuring reading time and comprehension, would turn ratings into evidence about reading.

Limitations.

- **One reader with dyslexia**, who rated rewrites on a web page; nobody with dyslexia has yet used the app, and the team raters do not have dyslexia.
- **The meaning scorer is a model**, stricter than people (it fails 53 of the 194 outputs they kept) and checked on dev outputs of the v0.2 systems.
- **Ease and Arabic are weakly agreed** between raters (α 0.19 and 0.17), so ease is measured through clause length, reading level and hard words, and the clause measure credits added commas as well as splits.
- **BayanSimplify-v0.3's meaning rests on the judge**. No significant difference is not proof of none: its interval against BayanSimplify-v0.2 on test runs from -7, +1 points, and its human round covers 18 sentences.
- **Some tracks are small**. E3 and E4 hold 64 and 40 test items, and few core items contain a number, so these rates separate only large differences.
- **No reference-based test score for the v0.2 models**: their test evaluation is BayanBench, which has no human references.
- **One run per configuration, one phone**. Each model was trained once, with one seed, so the comparison between architectures carries no estimate of training variance; on-device figures come from one mid-range phone.

Conclusion. Bayan shows that Arabic text can be simplified entirely on a mid-range phone, from inside any app, at interactive latency, and that the reduction in linguistic load can be measured honestly. Getting there depended less on model size than on evaluation: the field's standard score hid both what a model had learned and what it broke, and the behaviours a reader needs had to be measured one by one before they could be improved. Measured that way, BayanSimplify-v0.3 with four beams is the best model we built, and it is the one the app ships.

References

- [1] A. F. Aldakhil. Prevalence of developmental dyslexia among primary school children in Arab countries: a systematic review and meta-analysis. *Research in Developmental Disabilities*, 152:104812, 2024. doi:10.1016/j.ridd.2024.104812
- [2] E. Saiegh-Haddad and H. Taha. The role of morphological and phonological awareness in the early development of word spelling and reading in typically developing and disabled Arabic readers. *Dyslexia*, 23(4):345–371, 2017. doi:10.1002/dys.1572

- [3] L. Rello, R. Baeza-Yates, L. Dempere-Marco and H. Saggion. Frequent words improve readability and short words improve understandability for people with dyslexia. *INTERACT*, pp. 203–219, 2013. doi:10.1007/978-3-642-40498-6_15
- [4] M. Rivero-Contreras, P. E. Engelhardt and D. Saldaña. An experimental eye-tracking study of text adaptation for readers with dyslexia: effects of visual support and word frequency. *Annals of Dyslexia*, 71(1):170–187, 2021. doi:10.1007/s11881-021-00217-1
- [5] I. A. Asadi, A. Asli-Badarneh and V. Vaknin-Nusbaum. The impact of morphological density on reading comprehension in Arabic: a comparison between typically developing and children with reading disabilities. *Dyslexia*, 30(1):e1761, 2024. doi:10.1002/dys.1761
- [6] L. Javourey-Drevet, S. Dufau, T. François, N. Gala, J. Ginestíe and J. C. Ziegler. Simplification of literary and scientific texts to improve reading fluency and comprehension in beginning readers of French. *Applied Psycholinguistics*, 43(2):485–512, 2022. doi:10.1017/S014271642100062X
- [7] E. W. Hermena, D. Drieghe, S. Hellmuth and S. P. Liversedge. Processing of Arabic diacritical marks: phonological–syntactic disambiguation of homographic verbs and visual crowding effects. *JEP: Human Perception and Performance*, 41(2):494–507, 2015. doi:10.1037/xhp0000032
- [8] A. Madjidi and C. Crick. Enhancing textual accessibility for readers with dyslexia through transfer learning. *ASSETS*, pp. 1–5, 2023. doi:10.1145/3597638.3614473
- [9] M. Gharaibeh and Z. Basulayyim. ChatGPT enhances reading comprehension for children with dyslexia in Arabic language. *Disability and Rehabilitation: Assistive Technology*, 20(8):2858–2871, 2025. doi:10.1080/17483107.2025.2508389
- [10] B. Alhafni, R. Hazim, J. D. Piñeros Liberato, M. Al Khalil and N. Habash. The SAMER Arabic Text Simplification Corpus. *LREC-COLING*, pp. 16079–16093, 2024.
- [11] K. Elmadani, N. Habash and H. Taha. A Large and Balanced Corpus for Fine-grained Arabic Readability Assessment. *Findings of ACL*, pp. 16376–16400, 2025.
- [12] Baseet: multi-level Arabic text simplification with Aya-8B-generated targets and AraT5v2/AraBART baselines. Graduation project, University of Jeddah, 2026. <https://github.com/raya-y/Baseet>
- [13] DAASI: parallel Arabic simplification of government and insurance text, GPT-4o drafts verified by two annotators. tabsIT project, CCO. <https://github.com/Huseen1141/tabsit>
- [14] T. Zerrouki and A. Balla. Tashkeela: Novel corpus of Arabic vocalized texts, data for auto-diacritization systems. *Data in Brief*, 11:147–151, 2017.
- [15] C. Scarton and L. Specia. Learning simplifications for specific target audiences. *ACL*, pp. 712–718, 2018. doi:10.18653/v1/P18-2113
- [16] L. Martin, É. de la Clergerie, B. Sagot and A. Bordes. Controllable sentence simplification. *LREC*, 2020.
- [17] L. Martin, A. Fan, É. de la Clergerie, A. Bordes and B. Sagot. MUSS: Multilingual unsupervised sentence simplification by mining paraphrases. *LREC*, pp. 1651–1664, 2022.
- [18] W. Xu, C. Napoles, E. Pavlick, Q. Chen and C. Callison-Burch. Optimizing statistical machine translation for text simplification. *TACL*, 4:401–415, 2016.
- [19] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi. BERTScore: Evaluating Text Generation with BERT. *ICLR*, 2020.
- [20] F. Alva-Manchego, C. Scarton and L. Specia. The (un)suitability of automatic evaluation metrics for text simplification. *Computational Linguistics*, 47(4):861–889, 2021. doi:10.1162/coli_a_00418
- [21] A. Devaraj, W. Sheffield, B. C. Wallace and J. J. Li. Evaluating factuality in text simplification. *ACL*, pp. 7331–7345, 2022. doi:10.18653/v1/2022.acl-long.506
- [22] L. Cripwell, J. Legrand and C. Gardent. Simplicity level estimate (SLE): a learned reference-less metric for sentence simplification. *EMNLP*, pp. 12053–12059, 2023. doi:10.18653/v1/2023.emnlp-main.739
- [23] D. Heineman, Y. Dou, M. Maddela and W. Xu. Dancing between success and failure: edit-level simplification evaluation using SALSA. *EMNLP*, pp. 3466–3495, 2023. doi:10.18653/v1/2023.emnlp-main.211
- [24] M. T. Ribeiro, T. Wu, C. Guestrin and S. Singh. Beyond accuracy: behavioral testing of NLP models with CheckList. *ACL*, pp. 4902–4912, 2020.
- [25] A. M. Bean et al. Measuring what matters: construct validity in large language model benchmarks. *NeurIPS Datasets and Benchmarks*, 2025.

- [26] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy and M. J. Kochenderfer. BetterBench: assessing AI benchmarks, uncovering issues, and establishing best practices. *NeurIPS Datasets and Benchmarks*, 2024.
- [27] E. Miller. Adding error bars to evals: a statistical approach to language model evaluations. arXiv:2411.00640, 2024.
- [28] D. Li et al. Preference leakage: a contamination problem in LLM-as-a-judge. *ICLR*, 2026.
- [29] K. Krippendorff. *Content Analysis: An Introduction to Its Methodology*, 4th ed. SAGE, 2019.
- [30] Google DeepMind. Gemma 4, 31B instruction-tuned, quantisation-aware int4 checkpoint (google/gemma-4-31B-it-qat-w4a16-ct). Hugging Face, 2026.
- [31] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, pp. 611–626, 2023.
- [32] E. M. B. Nagoudi, A. Elmadany and M. Abdul-Mageed. AraT5: Text-to-Text Transformers for Arabic Language Generation. *ACL*, pp. 628–647, 2022.
- [33] A. Elmadany, E. M. B. Nagoudi and M. Abdul-Mageed. Octopus: A Multitask Model and Toolkit for Arabic Natural Language Generation. *ArabicNLP*, pp. 232–243, 2023.
- [34] M. K. Eddine, N. Tomeh, N. Habash, J. Le Roux and M. Vazirgiannis. AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization. *WANLP*, pp. 31–42, 2022.
- [35] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140):1–67, 2020.
- [36] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *ACL*, pp. 7871–7880, 2020.
- [37] D. Samuel, A. Kutuzov, S. Touileb, E. Velldal, L. Øvrelid, E. Rønningstad, E. Sigdel and A. Palatkina. Nor-Bench – A Benchmark for Norwegian Language Models. *NoDaLiDa*, pp. 618–633, 2023.
- [38] L. Burchell et al. An Expanded Massive Multilingual Dataset for High-Performance Language Technologies (HPLT). *ACL*, pp. 17452–17485, 2025.
- [39] O. Obeid et al. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *LREC*, pp. 7022–7032, 2020.
- [40] abjadai. CATT: Character-based Arabic Tashkeel Transformer. <https://github.com/abjadai/catt>
- [41] T. Zerrouki. Mishkal: Arabic text vocalization. <https://github.com/linuxscout/mishkal>
- [42] Libtashkeel, an ONNX Arabic diacritization model (MIT), via the text2tashkeel wrapper.

5. Team Member Review and Comment



**Marwan
Elamami**



**Abdulrahman
Khengari**



**Ahmed
Alaeb**



**Sanad
Ali**



**Mohammed
Thabet**



**Abdul Majid
Mraied**

Team Cogni

NAME	REVIEW and COMMENT
Marwan Elamami	Leading Bayan taught me that the hard part of an AI product is knowing whether it works. I am proudest that we built a benchmark for the task, checked it against people, and let the numbers change our decisions, which is how BayanSimplify-v0.3 with four beams became the app's large model. Next, I want readers with dyslexia to use the app and to measure their reading itself.
Abdulrahman Khengari	I built the data side, from the first synthetic corpus to corpus v1, the meaning judge and BayanSimplify-v0.3, and the benchmark report that compared 35 systems on the same rules. Measuring every variant the same way is what showed that BayanSimplify-v0.3 with four beams is our best model.
Ahmed Alaeb	Training BayanSimplify-v0.1 and the v0.2 models showed me how much the data decides: SAMER taught word swaps, while the corpora we built taught the model to restructure. I am glad the model work ended in an app that runs offline on an ordinary phone.
Sanad Ali	I benchmarked the diacritizers and chose Libtashkeel, which now adds tashkeel in the app without changing a letter, and trained BayanSimplify-v0.2-Fast (AraBART), whose speed made it the fast default.
Mohammed Thabet	Building the Android side, from the selection menu and the result sheet to model downloads and typography settings, showed me how much a reading tool depends on small details: spacing, clear states, and never losing the reader's place.
Abdul Majid Mraied	My part was the evaluation groundwork: scoring and leakage scripts, the locked test sets and the copy baselines. They make sure a score in this report measures what it claims to.

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	__/10	
APPLICATION	__/30	
RESULT	__/30	
PROJECT MANAGEMENT	__/10	
PRESENTATION & REPORT	__/20	
TOTAL	__/100	